

Lecture 1: Organising Multivariate Data

STAT 481 — Multivariate Analysis

Tue, Aug 25, 2026 • 50 minutes

Today's Overview

- **Today's question, on real data:** forty-one American cities, seven measurements each — *which city is unusual, and unusual how?*
- Build the object every method in this course operates on: the $n \times p$ **data array**, its rows (units) and its columns (variables).
- **The first R of the course:** load a real data set, and ask it the three questions you ask any data set before analysing it.
- Learn the one fork that decides which method you will ever use: *interdependence* or *dependence*.
- **Next:** on Thursday we compress those 287 numbers into a handful.

Packages today

- HSAUR2 [NEW] — supplies USairpollution, the 41-city data set of Lectures 1, 2, 4 and 5, back again for principal components in November.
- base / utils [known] — `dim()` (from base), `str()` and `head()` (from utils): the three questions you ask any data set before you analyse it. Both packages start with R; nothing to install and nothing to load.
- base [known] — `sort()`, `order()`, `rank()` and `pmin()`, in Section 3: what turns seven columns measured in seven different units into seven columns you can compare. They come back in Lecture 4 and in Worksheet 1.

Install it once by typing `install.packages("HSAUR2")` in the R console. After that, `library(HSAUR2)` is all you need, once per session.

Learning Objectives

By the end of this lecture, you should be able to:

- Explain, with an example, why a multivariate question cannot be answered one variable at a time.
- Write a data set as an $n \times p$ array and identify n , p and the entry x_{ik} for a stated unit and variable.
- Use `dim()`, `str()` and `head()` in R to establish the shape, the variable types and the first rows of a data set, and interpret what each prints.
- Extract one row, one column and one cell of a data frame.
- Say what `sort()`, `order()` and `rank()` each return *and what ranking a column is for*; compute a rank by hand from `#{below}` and the tie group; read a tie off a rank column and say how many values it involves; state which end carries rank 1 in a given table; and say why `pmin()` is not `min()`.
- Classify a research question as *interdependence* or *dependence*, and say which family of methods it sends you to.

1 Motivation: which city is unusual?

Here is a data set that has taught multivariate analysis for forty years, and it will follow us for three weeks. It records, for **41 cities in the United States**, seven numbers:

In R	What it is	Units
S02	sulphur dioxide in the air	micrograms per cubic metre
temp	average annual temperature	degrees Fahrenheit
manu	manufacturing enterprises with 20+ workers	count
popul	population, 1970 census	thousands of people
wind	average annual wind speed	miles per hour
precip	average annual rainfall	inches
predays	average number of days with rain per year	days

That is $41 \times 7 = 287$ numbers. Here is the question.

The question of the next three weeks

Which of these 41 cities is unusual — and unusual *how*?

Why one column at a time is not an answer

Your first instinct is a good one: look at each column and find its extreme value. Let us try it. Rank the cities on three of the seven variables — rank 1 is the largest — and look at five cities:

	S02	precip	popul
Chicago	1.0	29	1
Phoenix	36.5	41	16
Albuquerque	33.5	40	33
Providence	2.0	12	35
Miami	36.5	1	29

That table is R output, and you are not expected to be able to write it yet: the command that builds it, `rank()`, is taught in Section 3, and the lines that assemble the table are the `rank`s chunk of `L1.STAT481.qmd` in the course project, for afterwards.

Why is a rank a half? Four cities tie at exactly 10 units of sulphur dioxide, so between them they fill ranks 35–38; each is given the average, 36.5, rather than an arbitrary order. A half in a rank column always means a tie — Section 3 gives the rule, the command, and the case the rule does *not* catch.

Read those five rows slowly, because the entire course is in them.

- **Chicago** is rank 1 on sulphur dioxide and rank 1 on population. Column-by-column screening finds it immediately — but Chicago is the *easy* case, and we did not need a course for it.
- **Phoenix** is rank 36.5 on pollution (i.e. among the cleanest), rank 41 on rainfall (the driest of all 41) and rank 16 on population — thoroughly ordinary. *No single column flags Phoenix as extreme.* And yet a hot, very dry, mid-sized city is a genuinely unusual combination in this data set. Column-by-column screening misses it.
- **Providence** is rank 2 on pollution and rank 35 on population: a *small* city with *almost the worst* air. Neither number is remarkable on its own — rank 2 out of 41 is high but not shocking, rank 35 is unremarkable — but the two *together* are strange, and they are strange in a way that no column contains.

Fact 1.1: Why this course exists

An observation can be unremarkable on *every variable separately* and still be a very unusual *combination* of them. Screening one column at a time cannot find it, because the information is not in any column — it is in the relationship between them.

Multivariate analysis is the set of methods that treat the p measurements on a unit as **one object** rather than as p separate numbers.

That is the promise and the schedule: today we build the object, Thursday we summarise it, and on **Thursday Sep 3** we finally get a defensible answer to “which city is unusual” — which will not be the obvious one.

2 The data array

Every method in this course starts from the same object: a rectangle of numbers with one row per unit and one column per variable.

Notation

Throughout STAT 481: **$\mathbf{X}$** is the $n \times p$ **data matrix**. Its rows are the **units** (also called observations, individuals, items or cases) and its columns are the **variables**. We index units by $i = 1, \dots, n$ and variables by $k = 1, \dots, p$, so

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}, \quad x_{ik} = \text{value of variable } k \text{ on unit } i.$$

Matrices are **bold upright capitals**; vectors are **bold lowercase**; an individual observation is the column vector $\mathbf{x}_i = (x_{i1}, \dots, x_{ip})^\top \in \mathbb{R}^p$.

Read that last line as “ $\mathbf{x}_i$ is unit i ’s p measurements, standing up as a column”: the symbol $^\top$ (*transpose*) is what tips a row on its side, and $\in \mathbb{R}^p$ says only “ p numbers”. Both are unpacked in Lecture 6; today you need neither.

	var 1	var 2	...	var k	...	var p	
unit 1							n rows units (41 cities)
unit 2							
unit i				x_{ik}			
$\vdots$							
unit n							
	p columns — variables (7 measurements)						

△! The two questions to ask about any data set

Before anything else: **what is a row?** and **what is a column?** If you cannot answer both in one sentence each, you are not ready to compute anything. In *USairpollution* a row is *a city* and a column is *a measurement*. If someone hands you a table where a row is a city–year pair, your n and your p are not what you think they are.

Not every column is a number (read at home — Home Exercise 1(b) tests it)

The seven columns above are all **metric**: they are measured on an interval or ratio scale — the gaps between values mean something — so subtracting and averaging them is legitimate. A column such as “region of the country” or “coastal / inland” would be **non-metric** — a label, not a quantity — and averaging it would be meaningless. Almost everything in STAT 481 assumes metric variables; the exceptions are the grouping labels of Lecture 18 (MANOVA) and Lecture 26 (clustering), which we explain or recover, never average.

3 The first R of the course

Everything from here on is done in R. Open the course project, `STAT481.Rproj`, and type along. Every function is explained the first time it appears, and nothing below assumes you have programmed before.

Two rules before the first line. (1) Anything after a `#` is a *comment*: R ignores it completely. It is there for you, not for the machine, and you never need to type it. (2) Quotation marks mark *text*. Bare `HSAUR2` is a name R already knows; `"HSAUR2"` in quotes is just six characters, which is what `install.packages()` and `data()` ask for. Copy the quoting exactly as printed.

Load the data, and ask it its shape

```
library(HSAUR2)           # make the HSAUR2 package's contents available
data("USairpollution")   # load the data set into this R session
dim(USairpollution)      # how many rows, how many columns?
```

```
[1] 41  7
```

Three new things, one per line.

- `library(HSAUR2)` makes the contents of an installed *package* available — a package being a bundle of functions and data someone else wrote. Install it once; `library()` it once per R session.
- `data("USairpollution")` copies that data set from the package into your session so you can work with it.
- `dim()` returns the dimensions of a rectangular object: **rows first, then columns**. The `[1]` at the start of the output is not part of the answer — it is R telling you that the printed line starts with the first element. The answer is 41 and 7.

So $n = 41$ and $p = 7$, exactly as promised: **X** is 41×7 .

Ask it what the columns are

```
str(USairpollution)      # "structure": what type is each column?
```

```
'data.frame':  41 obs. of  7 variables:
 $ S02      : int  46 11 24 47 11 31 110 23 65 26 ...
 $ temp     : num  47.6 56.8 61.5 55 47.1 55.2 50.6 54 49.7 51.5 ...
 $ manu     : int  44 46 368 625 391 35 3344 462 1007 266 ...
 $ popul    : int  116 244 497 905 463 71 3369 453 751 540 ...
 $ wind     : num   8.8 8.9 9.1 9.6 12.4 6.5 10.4 7.1 10.9 8.6 ...
 $ precip   : num  33.36 7.77 48.34 41.31 36.11 ...
 $ predays  : int  135 58 115 111 166 148 122 132 155 134 ...
```

`str()` is the most useful single function in R — run it on every data set you ever meet. It reports:

- the **object type** — here a `data.frame`, R's standard rectangular table (Lecture 3 explains how it differs from a *matrix*, and why we will care);
- the **shape** again, in words: 41 observations of 7 variables;
- one line per column, giving its **name**, its **type** (`int` = whole numbers, `num` = numbers with decimals) and its **first few values**. The `$` is how R refers to a named column.

Read the types. All seven columns are numeric, hence metric, so every method in this course applies without preparation. Real data is rarely this obliging.

Ask it to show you the top of the table

`head()` takes a *second* input after a comma — how many rows you want. Most R functions are like this: one required input, then optional extras.

```
head(USairpollution, 4) # the first 4 rows
```

	S02	temp	manu	popul	wind	precip	predays
Albany	46	47.6	44	116	8.8	33.36	135
Albuquerque	11	56.8	46	244	8.9	7.77	58
Atlanta	24	61.5	368	497	9.1	48.34	115
Baltimore	47	55.0	625	905	9.6	41.31	111

This is **X**, or rather its first four rows. Compare it with the array in Section 2: the city names down the left are the *row names* (they label the units, they are not an eighth variable), the seven names across the top are the variables, and each number in the body is one x_{ik} .

So $x_{2,6}$ — unit 2, variable 6 — is Albuquerque's rainfall, 7.77 inches. Hold on to it: it is the *second* smallest rainfall in the data set — Phoenix is drier still, at 7.05, as you will see in a moment — and on Thursday Sep 3 the pair of them is going to cause trouble.

Reach into the table: one row, one column, one cell

A data frame is indexed with square brackets, **row first, then column**, separated by a comma — the same order as `dim()`. Leaving a slot empty means “all of them”.

```
USairpollution["Albuquerque", ] # one row: all 7 variables, one city
```

	S02	temp	manu	popul	wind	precip	predays
Albuquerque	11	56.8	46	244	8.9	7.77	58

```
USairpollution["Phoenix", "precip"] # one cell: both names given
```

```
[1] 7.05
```

And a whole column has its own shorthand, the dollar sign you saw in `str()`: `USairpollution$S02` is that column, all 41 values. `which.max()` then gives the *position* of the largest — a position, not a name — so this takes two steps, and it is worth seeing them apart:

```
which.max(USairpollution$S02) # which row holds the largest S02?  
rownames(USairpollution)[7]  # so ask for the 7th row name
```

```
[1] 7  
[1] "Chicago"
```

Look hard at the second line. `rownames()` gives a plain list of 41 names, and a list is indexed with *one* number and **no comma**: the comma rule above is for the two-dimensional table only. Now nest them. R works from the inside out — it computes `which.max()` first and hands the 7 to the brackets:

```
rownames(USairpollution)[which.max(USairpollution$S02)]  
max(USairpollution$S02)
```

```
[1] "Chicago"  
[1] 110
```

Two commands, so two answers, in that order. Chicago has the most sulphur dioxide, 110 micrograms per cubic metre — and, as we saw at the start of the lecture, not an answer to the question we actually asked.

Put a column in order: `sort()`, `order()`, `rank()`

Section 1 opened with a table of *ranks* and promised you the commands. Here they are. Three functions act on one column and return three different things. Telling them apart costs two minutes now and saves an hour in every lecture after this one.

First, what a rank is *for*. The seven columns are in seven different units, and you cannot compare 55.8 degrees with 608.6 thousand people — the two numbers are not measuring anything in common. But you *can* compare “12th of 41” with “17th of 41”. Replacing a value by its rank throws away the units, and that is the entire point: it is what lets the seven columns be put side by side. It also assumes almost nothing — only that the values in a column can be put in order. No centre, no spread, no distribution, nothing anyone can argue with.

Start with the sulphur dioxide of the first five cities — few enough to check by eye:

```
# Albany, Albuquerque, Atlanta, Baltimore, Buffalo
v <- USairpollution$S02[1:5]
v
```

```
[1] 46 11 24 47 11
```

```
sort(v)      # the VALUES, rearranged smallest to largest
order(v)     # the POSITIONS, in the order that would sort v
rank(v)      # for each value, WHERE IT STANDS -- left where it was
```

```
[1] 11 11 24 46 47
[1] 2 5 3 1 4
[1] 4.0 1.5 3.0 5.0 1.5
```

Three outputs, three different things. Take them one at a time.

- `sort(v)` returns the five **values**, increasing. It has thrown away who is who: 46 now sits fourth, and nothing in that line says it was Albany’s.
- `order(v)` returns **positions**: “to put this column in order, take entry 2 first, then entry 5, then 3, then 1, then 4.” Entries 2 and 5 are Albuquerque and Buffalo, the two smallest. So `v[order(v)]` is exactly `sort(v)` — and that is not a curiosity, it is the only way to sort a whole *table*.
- `rank(v)` returns **standings**, one per value, each left where it was: Albany’s 46 is the 4th smallest of the five, so the first entry is 4. Of the three, `rank()` is the only one whose output still lines up with the column it came from — which is why it, and not the other two, is what makes a temperature comparable with a population.

Positions are exactly what the row slot of `[row, column]` wants, so `order()` is how you sort a table by one of its columns. A minus sign flips the column, and sorting a flipped column upwards puts the largest first:

```
head(USairpollution[order(-USairpollution$S02), ], 3) # the three worst
```

	S02	temp	manu	popul	wind	precip	predays
Chicago	110	50.6	3344	3369	10.4	34.44	122
Providence	94	50.0	343	179	10.6	42.75	125
Philadelphia	69	54.6	1692	1950	9.6	39.93	115

Ties, and what a fractional rank does *not* tell you

Albuquerque and Buffalo both record `S02 = 11`, the two smallest of the five. Between them they occupy standings 1 and 2, and there is no honest way to say which comes first — so R gives each of them the *average*, 1.5. That is the default, `ties.method = "average"`, and it is the only one this course uses.

The rule is worth stating exactly, because it is not what people guess. If m values tie and between them they occupy standings $a, a + 1, \dots, a + m - 1$, every one of them is given the average of those standings,

$$a + \frac{m - 1}{2}.$$

So a shared rank is a **half when m is even** and a **whole number when m is odd**, and nothing else can ever appear.

Putting the two pieces together gives one formula that computes any rank straight from the column, with no sorting and no bookkeeping:

$$r_i = \underbrace{\#\{j : x_j < x_i\}}_{\text{how many are below you}} + \underbrace{\frac{1 + \#\{j : x_j = x_i\}}{2}}_{\text{half of one-plus-your-tie-group}}.$$

If nothing ties with x_i its tie group is x_i alone, the second term is 1, and r_i is the plain position $\#\{\text{below}\} + 1$. Check it on four numbers, $x = (7, 2, 9, 2)$, before trusting it on 41:

x_i	$\#\{\text{below}\}$	tie group	r_i
7	2 (the two 2s)	size 1	$2 + \frac{1+1}{2} = 3$
2	0	size 2	$0 + \frac{1+2}{2} = 1.5$
9	3	size 1	$3 + \frac{1+1}{2} = 4$
2	0	size 2	1.5

```
x <- c(7, 2, 9, 2)
x
rank(x)
```

```
[1] 7 2 9 2
[1] 3.0 1.5 4.0 1.5
```

Two consequences of the tie rule, both of which catch people out:

- A half means a tie, but **not necessarily a tie of two**. Four cities record exactly 10 units of sulphur dioxide; they fill standings 4 to 7 and all four share 5.5.
- A whole number **does not prove there is no tie**. Cleveland, Dallas and Omaha all record a wind speed of 10.9; they fill standings 35 to 37 and each is given 36, which looks like an untied rank and is not one.

```
so2 <- USairpollution$SO2           # a bare column: no names attached
names(so2) <- rownames(USairpollution) # give it the city names
wind <- USairpollution$wind
names(wind) <- rownames(USairpollution)

sum(so2 == 10)           # how many cities tie at 10 ...
rank(so2)[so2 == 10]     # ... and the HALF an even-sized tie gives
rank(wind)[wind == 10.9] # an odd-sized tie: a WHOLE number, still a tie
```

```
[1] 4
Houston Memphis      Miami Phoenix
      5.5      5.5      5.5      5.5
Cleveland   Dallas      Omaha
      36      36      36
```

If you actually need to know whether a value is tied, count it — do not read it off the rank. `sum(wind == 10.9)` answers the question; the 36 does not.

△! Rank 1 — but at which end?

`rank()` gives **rank 1 to the smallest** value. The table in Section 1 of these notes gives **rank 1 to the largest**: Chicago, with the most sulphur dioxide of the 41 cities, is rank 1 there and rank 41 here. Neither is wrong. They are one ordering read from opposite ends, and for n values

$$r_{\text{desc}} = n + 1 - r_{\text{asc}}.$$

In R you get the descending version by ranking the negatives, and the identity is checkable in one line:

```
n <- nrow(USairpollution)
rd <- rank(-so2)                # rank 1 = LARGEST (the Section 1
                                end)
rd["Chicago"]
all.equal(rd, n + 1 - rank(so2)) # the same thing, computed two ways
```

```
Chicago
      1
[1] TRUE
```

This is also where Section 1's 36.5 came from: the four cities tied at 10 are the four *cleanest*, so counted from the dirty end they fill standings 35 to 38 and share 36.5 — the same tie, the same rule, the other end.

Never write “rank 1”, or say it aloud, without saying which end you mean. A good deal of the confusion in the next three weeks starts here.

△! min collapses; pmin compares

What it is for. Sooner or later you will want to answer a question *once per entry* rather than once for the whole table — for example, “how far is this value from the nearer end of its own column?”, which is the first thing Worksheet 1 asks of a rank table. A function that returns a single number cannot do that, and `min` is exactly such a function.

The analogy, and it is the whole idea. R splits every reduction in two, and you already know one half of the split without ever having thought about it:

	collapses to one number	one answer per entry
addition	<code>sum(a, b)</code>	<code>a + b</code>
minimum	<code>min(a, b)</code>	<code>pmin(a, b)</code>
maximum	<code>max(a, b)</code>	<code>pmax(a, b)</code>

Nobody has ever expected `a + b` to come back as a single number. **`pmin` is to `min` exactly what `+` is to `sum`**; the `p` stands for *parallel*, meaning the two inputs are walked side by side. In symbols, for two arrays A and B of the same shape,

$$\min(A, B) = \min_{i,k} \{a_{ik}, b_{ik}\} \in \mathbb{R}, \quad [\text{pmin}(A, B)]_{ik} = \min\{a_{ik}, b_{ik}\},$$

so `min` lands on a single number and `pmin` lands on an array the same size as what went in. All four on three numbers:

```
a <- c(3, 9, 2)
b <- c(5, 1, 8)
sum(a, b)      # one number
a + b          # one per position -- and nobody finds this surprising
min(a, b)      # one number
pmin(a, b)     # one per position -- the same idea, for minima
```

△! min collapses; pmin compares (continued)

```
[1] 28
[1]  8 10 10
[1]  1
[1]  3 1 2
```

What it will be used for: folding a column in half. If r holds the ranks $1, \dots, n$ then $n + 1 - r$ holds the same ranks counted from the other end, and taking the smaller of the two *entry by entry* folds the column at its middle. At $n = 7$ the whole idea fits on three lines:

```
r <- 1:7
rbind(r = r, "8 - r" = 8 - r, e = pmin(r, 8 - r))
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]
r	1	2	3	4	5	6	7
8 - r	7	6	5	4	3	2	1
e	1	2	3	4	3	2	1

Rank 1 and rank 7 both come out at 1 — both are at an end — and rank 4 comes out at 4, dead centre. At $n = 41$ the fold is at 21. The symptom of reaching for the wrong one is easy to recognise: you asked for one answer per entry and R handed back a single number.

Self-check (1 minute)

Let $w \leftarrow c(8, 3, 8, 1)$ and $u \leftarrow c(5, 5, 5, 2)$.

- (a) What does `sort(w)` print? And `order(w)`?
- (b) What does `rank(w)` print, and what does the fractional part tell you?
- (c) `rank(u)` contains no fraction at all. Does that mean u has no ties?
- (d) What does `pmin(w, 5 - w)` print — and what would `min(w, 5 - w)` print instead?

(Answers: (a) [1] 1 3 8 8 and [1] 4 2 1 3. (b) [1] 3.5 2.0 3.5 1.0: the halves say the two 8s are tied and share standings 3 and 4. (c) No — [1] 3 3 3 1: three values tie, they fill standings 2, 3 and 4, and the average of those is the whole number 3. (d) [1] -3 2 -3 1, one number per entry, against [1] -3, one number for everything.)

△! Rows are observations in R; the textbook writes an observation as a column

This is the single most common source of dimension errors in multivariate analysis, and it starts here.

In R, `USairpollution` has 41 rows and 7 columns: one *row* per city. **In Johnson & Wichern** — and in these notes — a single observation $\mathbf{x}_i$ is written as a $p \times 1$ *column* vector, because that is what makes the matrix algebra of Lectures 6–9 come out right.

Both conventions are correct and they do not conflict: $\mathbf{x}_i^\top$ is row i of $\mathbf{X}$. In Lecture 6 (Sun Sep 13) you will meet the consequence, which has a name in this course: **the transpose trap**. Whenever R complains about “non-conformable arguments”, check this first.

Self-check (1 minute)

`USairpollution` is 41×7 . Suppose I add three more cities and delete the `wind` column.

- (a) What are the new n and p ?
- (b) What would `dim()` print?

Self-check (1 minute) (continued)

(c) What is $x_{3,2}$ in the data as printed above — the number, and what it measures?

(Answers: (a) $n = 44$, $p = 6$. (b) [1] 44 6. (c) 61.5 — Atlanta's average annual temperature in degrees Fahrenheit.)

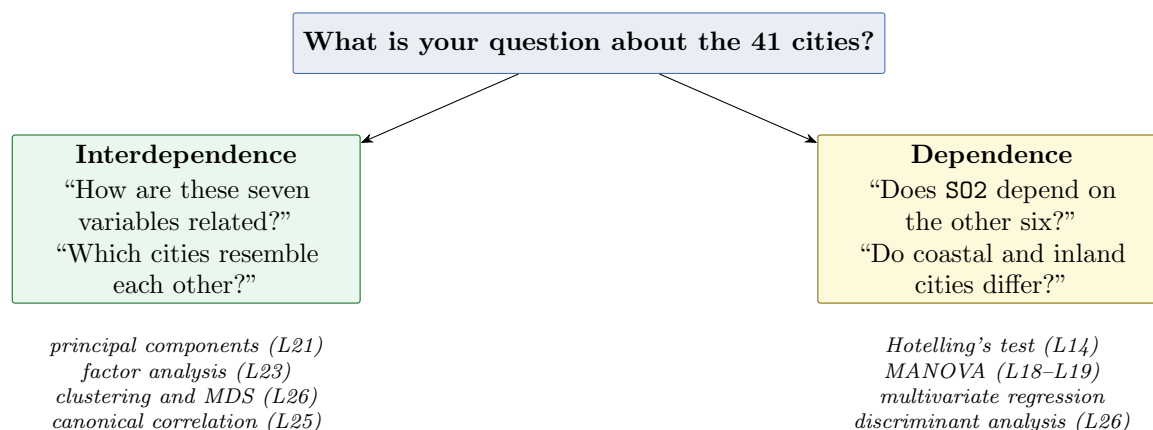
4 The fork that decides every method

There are a lot of multivariate methods, and they exist because questions come in two kinds. Deciding which kind you are asking is almost the first thing you do with a new data set, because it sends you down one of two branches — and you will never need the other one for that question.

Definition 4.1: Interdependence and dependence

An **interdependence** question asks about the structure *among* a set of variables, or *among* a set of units, with no variable singled out. Nothing is being predicted; you are asking what the data set is shaped like.

A **dependence** question singles out one or more variables as **responses** to be explained or predicted by the others.



Our question is an interdependence question. “Which city is unusual?” singles out no variable and predicts nothing; it asks about the shape of the whole cloud of 41 points. That is why everything up to Lecture 14 in October — summaries, pictures and distance — is interdependence work.

△! The fork is decided by your question, not by your data

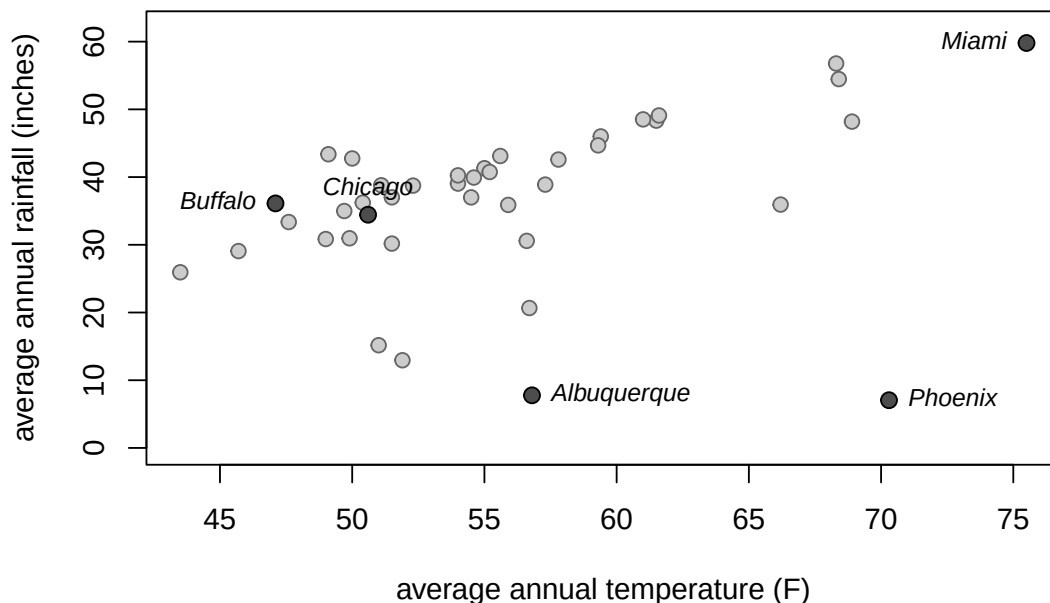
The same seven columns support both kinds of question. Nothing about `USairpollution` makes S02 a “response”; *you* make it one by asking whether it can be explained by the other six. Students routinely say “this is a regression data set” or “this is a PCA data set”. There is no such thing. There are regression *questions* and PCA *questions*.

5 One picture, before we have any formula

Two of the seven variables are about climate: temperature and rainfall. Plotting one against the other costs one line and already tells us something the ranking table could not.

```
plot(USairpollution$temp, USairpollution$precip,
     xlab = "average annual temperature (F)",
     ylab = "average annual rainfall (inches)")
```

That is *one* command spread over three lines: R keeps reading until the brackets close, so the indenting is only for your eyes. `plot()` takes the horizontal axis first and the vertical second, and `xlab=` and `ylab=` are *named* optional extras, which may be given in any order.



The one line above gives you all 41 points, plain and unlabelled — that is the correct result, not a mistake. The five dark labelled points were added afterwards; that code is in `L1-STAT481.qmd`.

Warmer cities are, on the whole, wetter cities — and then there are the exceptions. **Phoenix** and **Albuquerque** sit alone at the bottom: warm and almost rainless, which is what a desert climate looks like in this data set. **Miami** is at the far top right, the wettest and the warmest. **Buffalo** is cold and wet. And **Chicago** — the city that column-by-column screening found instantly, because of its sulphur dioxide — is sitting squarely in the middle of the crowd. On *these* two variables Chicago is utterly ordinary.

Fact 5.1: Two lessons from one scatterplot

1. The eye finds unusual points long before any formula does. That is why Lecture 4 (Tue Sep 1) is devoted to seeing multivariate data.
2. Which points look unusual *depends on which variables you plot*. Chicago is extreme on pollution and population, and completely ordinary on climate. With seven variables there are $\binom{7}{2} = 21$ such pictures — read $\binom{7}{2}$ as “the number of ways of choosing 2 things from 7” — and no guarantee that any of them shows the thing that matters. We will need something better than the eye — and building it is what Lectures 2 to 5 are for.

Wrap-up

Summary of Lecture 1

- An observation can be ordinary on **every variable separately** and still be an unusual **combination**. That is the whole reason this course exists — and Phoenix is the example to remember.
- Data is organised as the $n \times p$ array **X**: **rows are units, columns are variables**, and x_{ik} is variable k on unit i .
- Your question — not your data — decides whether you are doing **interdependence** or **dependence** work.
- **The R to remember:** `dim()`, `str()`, `head()` — run all three on every data set you are ever handed, before you compute anything. Then `sort()` (values), `order()` (positions, so it sorts a table) and `rank()` (standings, so it stays aligned with the column); and `pmin()`, which compares entry by entry where `min()` collapses to one number.
- **The trap to remember:** R stores an observation as a *row*; the textbook writes it as a $p \times 1$ *column*.

What you can do tomorrow: take any spreadsheet you are given, state its n and its p , say whether its columns are metric, and say whether your question is an interdependence or a dependence question.

Next lecture (Thu, Aug 27): *Summary Statistics*. We compress those 287 numbers into a mean vector and two 7×7 matrices — and find out which two of the seven variables are almost the same variable. Two new symbols get unlocked.

Announcements:

- **End of Add/Drop: Thursday, Aug 27, 2026** — at the end of the next session.
- **Lecture 3 (Sun, Aug 30) is the first R lab.** Bring your laptop with R, RStudio and Quarto *already installed*. If anything failed to install, see me before then.
- **R Lab 1 is out Sun, Aug 30** and is due **Sun, Sep 6**.
- **Quiz 1 is Thu, Sep 10**, covering Lectures 1–5.

Try at home (answers below; full solutions in the practice session)

Home Exercise 1. A hospital records, for each of 250 patients admitted in one month: age, systolic blood pressure, diastolic blood pressure, body-mass index, blood-glucose level, and the ward they were admitted to.

- What are n and p for the metric part of this data set?
- Which recorded column is not metric, and what would go wrong if you averaged it?
- Give one interdependence question and one dependence question about these patients.

Home Exercise 2. Run this. In the course project, load `USairpollution` and answer with R, reporting exactly what R printed:

- Which city has the *fewest* days of rain per year, and how many? (*Hint: `which.min()` works like `which.max()`.*)
- What is Chicago's average annual wind speed?
- Is the city from part (a) the same one that has the least rainfall? What does your answer say about screening one column at a time?
- Print the three *wettest* cities, whole rows, with one command. (*Hint: the `order()` pattern of Section 3, and mind the sign.*)

- (e) `rank(USairpollution$wind)["Cleveland"]` is a whole number. Does that settle whether Cleveland's wind speed is tied with anyone else's? Answer with a command, not with the rank.

Home Exercise 3. Before R Lab 1. Using only what is on these pages, write the single line of R that returns each of the following, then run it and report what R printed:

- (a) every measurement for Miami;
- (b) Miami's population only;
- (c) the name of the windiest city.

Answers: 1) (a) $n = 250$, $p = 5$ — age, systolic, diastolic, BMI, glucose. (b) Ward is a label, not a quantity: the “average ward” of a set of patients is meaningless, and if the wards happen to be numbered 1–6 R will cheerfully compute it anyway. (c) Interdependence: “which of these five measurements move together, and can they be summarised by fewer numbers?” Dependence: “do the five measurements differ between wards?” — which is Lecture 18. 2) (a) **Phoenix**, with 36 days — the fewest in the data set. (b) [1] 10.4 miles per hour. (c) Yes, in this instance: Phoenix has both the fewest rain-days (36) and the least rainfall (7.05 inches), with Albuquerque second on both — but that agreement is a fact about American deserts, not a general rule, as *SO2* and *popul* show for Providence. (d) `head(USairpollution[order(-USairpollution$precip), ], 3)` — Miami (59.80), New Orleans (56.77), Jacksonville (54.47). Drop the minus sign and you get the three driest instead. (e) No. A whole rank only says the tie, if there is one, has an odd number of members: `sum(USairpollution$wind == 10.9)` prints [1] 3, so Cleveland is tied with Dallas and Omaha, all three sharing standings 35–37 and therefore the whole number 36. Count the value; never read a tie off a rank. 3) (a) `USairpollution["Miami", ]`. (b) `USairpollution["Miami", "popul"]`, printing [1] 335. (c) the two-step pattern of Section 3, which prints [1] "Wichita": `rownames(USairpollution)[which.max(USairpollution$wind)]`.